

Relazione tecnica del progetto

Architettura, tecnologie e scelte progettuali

Gruppo Mu.To

A.A. 2025–26

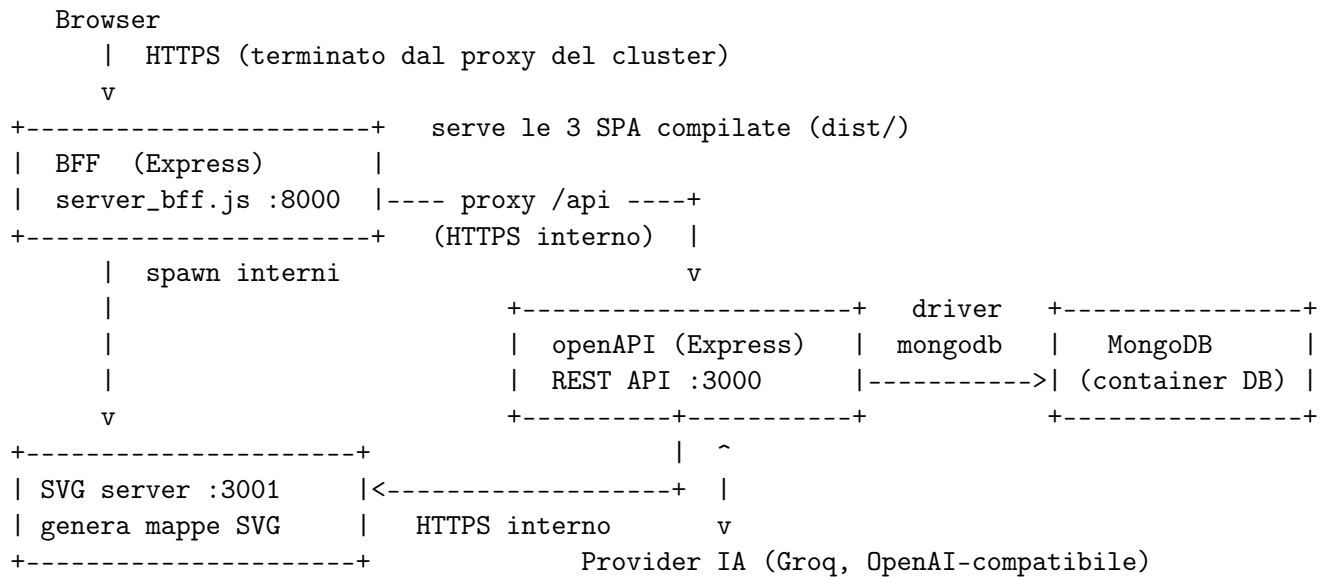
Questo documento descrive le *tecnologie*, l'*architettura* e la *struttura* del sito, motivando le scelte progettuali. Non descrive il dominio applicativo: l'obiettivo è spiegare *come* il sistema è costruito e *perché*, in funzione dei criteri di valutazione (generalità dei tool, flessibilità, usabilità, sofisticazione grafica).

Indice

1	Visione d'insieme dell'architettura	1
1.1	I componenti	1
2	Stack tecnologico e motivazione delle scelte (generalità)	2
2.1	Lato server	2
2.2	Lato client: tre SPA, due filosofie	2
3	Organizzazione del codice e flessibilità	3
3.1	Struttura dei sorgenti	3
3.2	Modello dei dati e delle annotazioni	3
3.3	Le collezioni MongoDB	3
3.4	Estendibilità	3
4	Comunicazione, autenticazione e sicurezza	3
4.1	Tre livelli di autorizzazione	3
4.2	TLS interno e terminazione esterna	4
5	Usabilità	4
6	Sofisticazione grafica	4
7	Integrazione dell'IA	5
8	Build e deployment	5

1 Visione d'insieme dell'architettura

Il sistema è organizzato a **livelli** (*tiered*), con una netta separazione tra dato, logica applicativa e presentazione. I componenti comunicano esclusivamente via HTTP/JSON, così che ciascuno sia sostituibile in modo indipendente.



1.1 I componenti

openAPI (server/openAPI)

È il cuore applicativo: una REST API in **Node.js** + **Express** che possiede la logica di dominio (musei, oggetti, percorsi, utenti, visite guidate, marketplace) e l'unico accesso a **MongoDB**. Espone inoltre l'integrazione con l'IA.

BFF (navigator/UI/bff)

Il *Backend-for-Frontend*: un secondo server Express che (a) serve le tre Single Page Application già compilate e (b) fa da **reverse proxy** verso l'API sul prefisso `/api`, iniettando la chiave applicativa. È l'unico processo esposto pubblicamente.

SVG server (map-creator/js_server)

Servizio dedicato che, a partire dal layout di un museo, costruisce la **mappa vettoriale SVG** delle sale; è isolato perché la generazione grafica è una responsabilità distinta dal resto della logica.

Le tre SPA

marketplace (catalogo, acquisti, visite), *viewer/navigator* (la visita vera e propria sulla mappa) ed *editor* (creazione/modifica dei musei).

Questa scomposizione è una scelta di **generalità**: ogni servizio fa una cosa sola, dialoga via JSON e potrebbe essere riscritto in un altro linguaggio senza toccare gli altri. Il BFF, in particolare, disaccoppia il browser dall'API: il client non conosce né l'indirizzo reale dell'API né la sua chiave.

2 Stack tecnologico e motivazione delle scelte (generalità)

2.1 Lato server

- **Node.js** + **Express** per API, BFF e SVG server: un unico runtime e un unico modello di routing su tutti i servizi server, che riduce il costo cognitivo e favorisce il riuso.
- **MongoDB** tramite il *driver nativo* (`mongodb`), non un ORM. La scelta è deliberata: i documenti dei musei hanno una forma annidata e variabile (sale, oggetti, descrizioni multidimensionali) che mal si presta a uno schema rigido. Il modello a documenti rende naturale rappresentare un museo come un singolo aggregato.

- **Elaborazione media e QR:** `sharp` per ridimensionare e normalizzare le immagini caricate, `qrcode` per generare i codici QR di sblocco contenuti, `multer` per gli upload *multipart*.
- **IA:** provider **Groq** con modello `llama-3.3-70b-versatile`, contattato tramite l'API *OpenAI-compatible*. Usare un'interfaccia standard (non un SDK proprietario) è una scelta di generalità: cambiare provider (OpenAI, Ollama locale, ...) richiede solo variabili d'ambiente, non codice.

2.2 Lato client: tre SPA, due filosofie

La parte più significativa è la scelta differenziata del livello di presentazione, calibrata sulla complessità di ciascuna applicazione.

- **viewer (navigator)** ed **editor** usano **React** (il viewer in **TypeScript**). Sono applicazioni a stato ricco e fortemente interattive — la navigazione sulla mappa SVG con *pathfinding*, la chat, l'editor di musei — dove un framework a componenti con stato reattivo e tipizzazione statica ripaga la sua complessità.
- **marketplace** usa **Alpine.js** + JavaScript “vanilla”. È stato *deliberatamente riscritto* per non dipendere da un framework pesante: la reattività dichiarativa di Alpine (`x-data`, `x-for`, `x-model`) copre esattamente i bisogni di una vetrina con filtri, modali e form, senza il peso di React. Il risultato è un bundle circa il 60% più leggero. Questo dimostra **uso corretto della tecnologia minima sufficiente** invece di adottare un framework per inerzia.
- **Build:** tutte e tre con **Vite**, usato solo come bundler/dev-server (non è un framework UI). In sviluppo, il proxy di Vite inoltra `/api` all'API iniettando la chiave, replicando il comportamento del BFF in produzione.

3 Organizzazione del codice e flessibilità

3.1 Struttura dei sorgenti

Ogni applicazione vive in una propria directory con sorgenti in `src/` e output di build in `dist/`. La separazione `src/dist` mantiene puliti i sorgenti versionati; il BFF serve i `dist/` come contenuto statico. La logica di rete (`api.js`) e la traduzione (`mpLocales.js`) sono moduli *framework-agnostic*: durante la migrazione del marketplace da React ad Alpine sono stati riutilizzati invariati, segno di un buon disaccoppiamento.

3.2 Modello dei dati e delle annotazioni

Un oggetto museale non ha una singola descrizione, ma una **matrice di descrizioni** indicizzata su due dimensioni:

- **livello** del visitatore (bambino, studente, esperto, avanzato);
- **durata**/profondità desiderata (breve, media, lunga, estesa).

A ciò si aggiunge `descrizioniI18n`, che replica la matrice per lingua (**IT/EN/FR**). Questa struttura è la chiave della **flessibilità del modello di annotazione**: lo stesso oggetto si adatta automaticamente al profilo dell'utente e alla lingua, e aggiungere un livello o una lingua è un'estensione dei dati, non una modifica del codice. Gli oggetti hanno inoltre un campo `connessi` che definisce un **grafo** di adiacenze fra opere, usato per calcolare percorsi (visita una sala alla volta, minimizzando gli spostamenti).

3.3 Le collezioni MongoDB

Il dato è partizionato per responsabilità: `musei` e `musei_layout` (struttura e mappa), `users` e `sessions` (identità), `professor_codes` e `qr_codes` (autorizzazioni), `guided_visits` (visite di

classe), `marketplace_object_requests` (richieste d'acquisto), `oggetti_immagini/stanze_immagini` (media). Le route REST riflettono questa organizzazione: gruppi `/musei`, `/users`, `/guided-visits`, `/admin`, `/ai`, `/qr`.

3.4 Estendibilità

Le scelte rendono il sistema facilmente estensibile: un nuovo *device* o browser è coperto dal layout responsive; un nuovo *modello di dati* (es. nuovo tipo di oggetto) si aggiunge come campo del documento; un nuovo *provider IA* o una nuova *lingua* sono configurazioni. La presenza di percorsi di *fallback* (vedi §7) garantisce che il sistema degradi in modo controllato anziché rompersi.

4 Comunicazione, autenticazione e sicurezza

4.1 Tre livelli di autorizzazione

1. **Chiave applicativa (X-API-Key)**: ogni richiesta all'API la deve portare. Il browser non la conosce: la inietta il BFF (o il proxy Vite in sviluppo). Protegge l'API a livello di trasporto.
2. **Sessione utente**: dopo il login l'API rilascia un cookie `auto_auth` **HttpOnly** con `SameSite=Lax`; il token è persistito in `sessions` con scadenza. Le password sono salvate come hash **scrypt** con salt, mai in chiaro.
3. **Ruoli**: `utente`, `professore`, `admin`. Il ruolo professore si ottiene in registrazione tramite un **codice** (verificato come hash SHA-256 lato server); abilita la creazione di visite guidate. L'admin accede alle funzioni di gestione del marketplace.

In aggiunta, contenuti specifici si **sbloccano via QR**: il viewer ha un componente `QrGate` che scansiona il codice fisicamente presente in museo.

4.2 TLS interno e terminazione esterna

L'API espone HTTPS anche internamente (il BFF vi si connette in HTTPS, fidandosi del certificato self-signed). Verso l'esterno, invece, sul cluster di dipartimento il **TLS è terminato dal proxy**: il BFF serve HTTP semplice sulla porta richiesta. Questa distinzione è gestita da configurazione (`BFF_FORCE_HTTP`), un esempio di adattabilità all'ambiente di deploy senza modifiche al codice.

5 Usabilità

L'attenzione all'utente che *non conosce* il modello applicativo si vede in diversi punti:

- **Internazionalizzazione completa** IT/EN/FR su tutte le interfacce, con la lingua legata al profilo e propagata via evento globale.
- **Adattamento al profilo**: descrizioni e percorsi si calibrano su livello e durata scelti, così l'utente vede contenuti adatti senza dover capire il modello sottostante.
- **Guida e feedback**: schermate di *briefing* a inizio percorso, *toast* non invasivi per esiti delle azioni, stati di caricamento (*skeleton*), e “guardie” che mostrano messaggi chiari quando manca l'autenticazione o il museo.
- **Attività altrui**: nelle visite guidate il professore vede in tempo reale lo stato degli studenti (dashboard), e gli studenti seguono la navigazione condivisa — consapevolezza delle azioni altrui.

- **Modalità d’accesso multiple:** chat in linguaggio naturale con l’IA (su singolo oggetto o sull’intero museo) e **comandi vocali** offline (riconoscimento con *Vosk*, eseguito interamente nel browser), che abbassano la barriera d’uso.

6 Sofisticazione grafica

La presentazione è governata da un **design system** coerente: un set di variabili CSS (palette, raggi, tipografia con i font *Cinzel* e *Raleway*) condiviso fra le SPA, che garantisce uniformità e rende banale un re-styling globale. Le scelte rilevanti rispetto al rapporto fra dati e maschere:

- **Mappa vettoriale:** la sede della visita è un SVG generato dal layout reale del museo, su cui il viewer disegna percorso, focus e spostamenti — la grafica è il dato spaziale, non una decorazione.
- **Differenziazione dei tipi:** oggetti “normali” e oggetti di solo testo, opere acquistabili e incluse, percorsi standard e percorsi IA personalizzati sono distinti visivamente (colore, etichette, badge), così che il tipo di dato sia leggibile a colpo d’occhio.
- **Rapporto maschera/dato:** liste con paginazione, gallerie immagini con miniature, tabelle delle descrizioni (livello $\times$ durata) presentano dati potenzialmente densi in maschere dimensionate al contenuto, con *label* comprensibili accanto ai valori formalizzati.
- **Responsive design:** griglie fluide e *breakpoint* adattano le maschere a desktop, tablet e mobile.

7 Integrazione dell’IA

L’IA è usata in modo *assistivo* e sempre con rete di sicurezza:

- **Percorsi personalizzati:** dato un museo e le preferenze dell’utente (interessi, livello, durata) e un eventuale **vincolo in linguaggio naturale** (“ho solo mezz’ora”, “siamo con bambini di 5 e 8 anni”), il modello seleziona e ordina le opere e genera testi di accompagnamento, tradotti nelle tre lingue.
- **Chat contestuale:** domande su un singolo oggetto o sull’intero museo, ancorate ai dati realmente presenti per ridurre le allucinazioni.
- **Robustezza:** ogni chiamata IA ha *timeout* e un *fallback* deterministico (selezione e descrizioni generate dai dati locali). Se il provider è irraggiungibile, l’utente ottiene comunque un percorso valido. È un esempio concreto di *flessibilità*: la feature degrada, non fallisce.

8 Build e deployment

La pipeline è semplice e riproducibile: `npm install` (le dipendenze non sono versionate) e `npm run build` compilano le tre SPA nei rispettivi `dist/`; `npm start` avvia il BFF, che a sua volta lancia i servizi interni (API e SVG server). In produzione sul cluster di dipartimento, il processo Node ascolta in HTTP sulla porta pubblica e il TLS è gestito a monte; MongoDB è un container raggiungibile solo dall’interno del cluster. La configurazione sensibile (chiavi, credenziali) vive in file `.env` non versionati, separando codice e segreti.

In sintesi. Il progetto privilegia la *tecnologia minima adeguata* a ogni compito (Alpine dove basta, React dove serve), una netta separazione di responsabilità fra servizi che comunicano via JSON, un modello dei dati pensato per adattarsi a profili, lingue e annotazioni diverse, e un

livello di presentazione coerente e consapevole del rapporto fra dati e maschere. Le soluzioni di compatibilità (HTTP/HTTPS, provider IA, chiave API) sono risolte per *configurazione*, non forzate nel codice.